

Remote runtimes: the CLI, the daemon and the index over TCP

Epic theseus-pp7c, a child of the north star theseus-3iwi. Written 2026-10-09 by Tabitha/Claude from the code at origin/main 69cb5375. A design only: nothing here is built yet. The owner took every default on 2026-10-09 (section 9), and the rows are filed (section 8). They start after the benchmark rerun (theseus-lc7a).

| The answer first

The ask (the owner, 2026-10-09): *"natively support theseusd and theseus-index possibly being fully remote from theseus cli and from each other -- so, the possibility that the core theseus runtimes are all TCP remotable".*

The design in five lines.

- **One framing, a new road.** The same newline JSON-RPC the Unix socket carries, inside rustls TLS 1.3 on TCP. The server and the client already take any stream, so the change is a listener and a connector.
- **Off by default, and local stays free.** `[rpc] listen` defaults to `["unix"]`. The Unix socket and stdio keep their code path, with no handshake and no TLS, so nothing local gets slower.
- **mTLS, not tokens.** A private CA held as `op://` refs; each client is a named device with a role (owner, observer, index); a connection is proven before any JSON is read, and no secret ever travels in a frame.
- **A remote client is its own surface.** `Surface::Remote`, never `Cli`: no host-path reads, a short list of local-only methods, every act ledgered with the device's name.
- **A remote index follows a WAL feed.** The daemon streams whole WAL frames (`wal.follow`) over one persistent mTLS connection; the tender keeps its cursor and index as today.

What it costs. At 1 ms of round trip nobody notices. At 30 ms, a cold `theseus ask` reaches its first event about 90 ms later (6% to 18% of a model's first token), a warm TUI 30 ms later, and recall still keeps about 220 ms of its 250 ms deadline. TLS adds tens of microseconds a frame at worst. Section 3 has the numbers.

The rule it changes. The spec's reachability rule, "Localhost only, outbound OK, no inbound", becomes "inbound TCP only with mTLS, and only when `[rpc] listen` names it" (decision D1, taken).

1. Every link between processes today

1.1 The CLI and the TUI to the daemon: a Unix socket, newline JSON-RPC

- **The listener.** `serve_socket` binds a `UnixListener` (`crates/theseusd/src/main.rs:1211-1212`), `chmods` it to `0600` (`main.rs:1213-1217`) and logs "localhost only, file permissions are the auth" (`main.rs:1220`).
 - Every accepted connection is `Surface::Cli` with no further check: "The socket is mode 0600: whoever connects is the CLI" (`main.rs:1252-1253`).
 - `SO_PEERCRED` is read only to log the peer's pid (`main.rs:1244-1246`). Nothing decides on it.
- **The socket's path.** `--socket` or `[server] socket` (`main.rs:96-98`, `main.rs:538`; `crates/theseus-core/src/config.rs:1094`, `config.rs:1696`). A stale socket is replaced only when nothing answers on it (`main.rs:1201-1210`).
- **stdio.** `--stdio` serves the same protocol on pipes (`main.rs:86-88`, `main.rs:564-597`), and the CLI's `--spawn` starts one as a child (`crates/theseus/src/client.rs:85-108`). A stdio daemon uses its own `store-stdio`, because the store is single-process (`main.rs:461-463`); it runs no index tender (`main.rs:1076-1079`) and binds no Discord (`main.rs:626-628`).
- **The framing.** `Core::serve_connection` takes any `AsyncRead/AsyncWrite` pair (`crates/theseus-core/src/rpc/server.rs:65-75`). It reads lines with an **unbounded** `lines()` (`server.rs:138`) and writes and flushes one line per message (`server.rs:419-421`).
 - One ordered outbound queue carries responses and notifications, capped at `BACKLOG_CAP = 4096` (`crates/theseus-core/src/outbound.rs:8-15`, `outbound.rs:27`).
 - Past the cap, notifications are dropped and `events.lost` names the streams to re-read (`crates/theseus-protocol/src/push.rs:301-314`).
- **The client side.** `Conn::socket` connects with a `UnixStream` (`client.rs:55-82`). `Conn::over` already takes any stream pair (`client.rs:110-125`), so a TLS stream plugs in without a new client. The TUI connects the same way (`crates/theseus-tui/src/main.rs:110-112`).
- **No version handshake.** `theseus_protocol::VERSION = "0.1"` (`crates/theseus-protocol/src/lib.rs:80`) is only reported inside `health` (`crates/theseus-core/src/rpc/methods.rs:52`; `lib.rs:488-493`).
- **Resume already exists, as a re-read.** `session.history` pages by WAL position, `after` and `before`, and "positions never move" (`crates/theseus-protocol/src/history.rs:11-25`). A reconnecting client calls `session.history after=<last>` and then `session.watch`.
- **theseus ask makes one round trip after connecting:** a single `turn.submit`, streamed (`crates/theseus/src/cmd.rs:64-81`, `cmd.rs:108-110`).

1.2 The daemon to the index tender: a child process and a Unix socket

- **Start and supervision.** The tender is the daemon's child, `theseus-index serve --store <dir> --index <dir> --parent <pid> ...` (`crates/theseus-core/src/tender.rs:330-345`). It starts 2 s after serving (`tender.rs:6-10`, `tender.rs:70-79`), only under the socket daemon (`main.rs:1076-1079`); it holds one lock per index directory (`tender.rs:11-14`); the reaper restarts it with a backoff from 1 s to 60 s

(`tender.rs:15-18`, `tender.rs:62-68`); a stop sends SIGTERM and `--parent` takes it down with the daemon (`tender.rs:19-22`); the binary is looked up only beside `theseusd` (`tender.rs:107-112`).

- **The socket.** The tender answers on `<state>/index/sock` (`tender.rs:308-311`). `call` opens a **new Unix connection per request** (`tender.rs:822-857`). Deadlines: health 100 ms (`tender.rs:80-83`), status 2 s (`tender.rs:84-85`), query 10 s plus the query's own wait (`tender.rs:86-87`).
- **What the tender reads.** The store's WAL, straight from the filesystem: `WalFollower::open(<store>/wal, cursor)` (`crates/theseus-index/src/tender.rs:345-357`), woken by **inotify** with a timer as the backstop (`crates/theseus-follow/src/lib.rs:36-37`). The follower's contract: whole frames only, a cut or a restore seen as a rewind, a safe read across rotations (`follow/src/lib.rs:6-35`). It learns each session's place from META records in the same WAL (`theseus-index/src/tender.rs:41-45`).
- **Recall bounds the index.** `[memory] recall_deadline_ms` defaults to 250 (`config/memory.rs:138`, `:185-186`). A late or missing answer is recorded `deadline` or `unavailable`, and the turn goes on (`crates/theseus-core/src/recall.rs:10-13`, `recall.rs:419-443`).
- **A second WAL reader already ships off the machine.** The durability tender follows the same WAL to S3, 5 to 60 s behind (`docs/spec/02-part1-s0.md:29`); `read_upto` keeps unsynced frames off the remote copy (`follow/src/lib.rs:28-34`).

1.3 The web UI and the cockpit: TCP, loopback only

- **Loopback is enforced twice:** `web::serve` refuses a bind that is not loopback (`crates/theseusd/src/web.rs:72-84`), and so does config validation (`config.rs:802`).
- **The browser checks.** Every request's `Host` must be the UI's own address, and a `/ws` upgrade's `Origin` the UI's own page, against DNS rebinding (`web.rs:8-15`, `web.rs:275-313`, `web.rs:381-416`).
- **The other-user check.** The connecting socket's owning uid must be the daemon's uid, read before any byte of the request (`web.rs:131-200`; `crates/theseus-core/src/peer.rs:1-14`, `peer.rs:72`), through `sock_diag` in about 12 μ s (`docs/spec/04-part1-s3.10.md:49`). There is no per-start token, by design (`web.rs:20-26`).
- **/ws is a bridge:** each text frame is one JSON-RPC line into an in-memory duplex served as `Surface::Web` (`web.rs:420-470`).

1.4 MCP, herdr and Discord

- **The MCP server** speaks streamable HTTP at `/mcp` on 127.0.0.1, behind a vault key and the same uid check (`crates/theseusd/src/mcp.rs:1-25`; `config/mcp_server.rs:18`). A client inside a job is recognized by process ancestry and carries the job's hold (`mcp.rs:20-23`): a check only possible on one machine.
- **herdr.** The adapter lives in the CLI and speaks to herdr's own local socket (`crates/theseus/src/herdr.rs:1-17`). It moves with the CLI; nothing in `theseusd` knows herdr.
- **Discord** runs inside the daemon and reads and writes the core directly (`docs/spec/04-part1-s3.10.md:320-331`). Its traffic is outbound only and unaffected.

1.5 What assumes one machine

- **The store is single-process**, and the tender reads its WAL by path. `restore` proves no daemon serves the store by connecting to the Unix socket (`main.rs:1297-1303`).
- **Host paths travel in RPC**. `place.publish` takes a `path` that the daemon reads on its own disk (`crates/theseus-core/src/rpc/publish.rs:153-166`), and the CLI sends the absolute path resolved against **its own** working directory (`cmd.rs:1377-1384`). From a remote CLI that is the wrong file, or a host-path read.
- **Uploads already safe for a remote CLI**: `ask --attach` sends the file's bytes, up to 32 MiB (`cmd.rs:1797-1830` ; `crates/theseus-core/src/attach.rs:1-35`), and `theseus import` streams the file's lines in batches (`crates/theseus/src/import.rs:133-136` ; `crates/theseus-protocol/src/import.rs:35-44`).
- **Tools run where the daemon runs**: `fs`, `shell`, the L1 sandbox and the job wrappers with their cgroups. Execution events carry the daemon-side `cwd` (`crates/theseus-protocol/src/events.rs:269`).
- **Health reads the daemon's own machine** (the disk under the state directory, `theseus-protocol/src/lib.rs:780-805` ; whether the binary is writable, `lib.rs:807-820`), so a remote reader must be told which host it describes.
- **Signals and systemd are local** (`main.rs:1230-1278`); `--spawn` only ever starts a local daemon.
- **The job marker**. Operator methods are refused inside a job when `THESEUS_SESSION` is set (`client.rs:262-300` ; `theseus-protocol/src/lib.rs:1858`): a speed bump, not a boundary (`peer.rs:10-14`). The L1 sandbox hides the daemon's socket from a job (`main.rs:553-554`).

2. What security rests on being local

1. **The rule itself**. "Localhost only, outbound OK, no inbound. External proxies handle exposure." (`docs/spec/02-part1-s0.md:58`); for the socket, "file permissions are the authentication" (`docs/spec/04-part1-s3.10.md:319`). Decision D1 amends it.
2. **0600 is the whole of CLI auth**. Anyone who can connect is "the CLI", the owner's own surface.
3. **Private places**. `Surface::reads_private` is true for `Cli` and `Web` only (`approval.rs:50-57`); a read with no target is a private place (`places.rs:271-274`); `owner_in_private` lets any non-Discord surface through (`places.rs:340-358`): confirms, trusts, untightens, publishes and the operator methods. So **any new surface mapped to cli would get the owner's full powers**. A remote surface must be its own variant.
4. **The peer's uid**. The web UI and the MCP server serve only the daemon's uid, read from the kernel's socket tables. That proof does not exist across machines.
5. **The owner's identity**. Implicit on local surfaces (`places.rs:164-168`); `[places] owner` as `discord:<id>` on Discord (`places.rs:51-60`). A remote client needs an explicit identity: a device and its role.
6. **The secret broker and the floor** resolve and judge on the daemon's host (`crates/theseus-core/src/broker.rs:1-20`). Neither changes, because tools stay on the daemon's machine.
7. **Jobs and the daemon's address**. A TCP listener on a routable address is reachable from any job with egress. Under mTLS that is safe only if **no client key ever sits on the daemon's host**.

3. Transport: NDJSON JSON-RPC over TLS 1.3 on TCP

- **The same bytes as the Unix socket, inside rustls.** The Unix socket stays the default and stays untouched.
 - **Why not /ws :** it adds HTTP upgrade and WebSocket framing, and its checks are for browsers. It stays the browser's transport, loopback only. The web UI is not made remote (D8).
- **Configuration.**
 - The daemon: `[rpc] listen = ["unix", "tls://10.0.0.5:7434"]`. The default is `["unix"]`: TCP off.
 - The client: `theseus --connect host:port, THESEUS_ADDR, or [client] connect`. `--connect` excludes `--spawn` and `--socket`.
 - The index: `[index] remote = "tls://host:7435"` replaces the child tender.
- **Handshake (hello).** On TCP the client's first frame is `hello { protocol: "1.x", build, client, features }`, pipelined with its first request, so no round trip is added. The daemon answers its protocol, build, host, features and time, and refuses a different major version with words that name both builds. The Unix socket and stdio do not require it. `VERSION` moves to semver; additive fields stay minor, as `serde` defaults already allow.
- **Frame cap.** Every transport gets a bounded line reader in place of `lines()`: 64 MiB, which covers a 32 MiB attachment in base64. An oversize line gets an error frame and the connection closes.
- **Streaming and backpressure, unchanged.** The per-connection queue, `BACKLOG_CAP` and `events.lost` hold over a slow link as they do for a frozen tab. TCP adds a write deadline (10 s with no progress closes the connection), a 15 s idle `ping`, `TCP_NODELAY` and `keepalive`.
- **Reconnect and resume.** Backoff from 0.5 s to 15 s; each watched session resumes with `session.history after=<last position>`, then `session.watch`. Positions never move, so nothing repeats and nothing is skipped. Turns keep running while a client is away; a confirm waits for any owner surface, as today.
- **Write coalescing, TCP only:** the writer flushes when the queue is empty, not after every line, so each flush is one TLS record.

3.1 FAST: what remote costs

- **The local path costs nothing new.** The only shared change is the bounded line reader, $O(1)$ per byte like `lines()`. Row R7's lifecycle-bench A/B proves it.
- **Per frame**, measured on the development machine with Python's `ssl` (TLS 1.3, one `sendall` per frame, 50,000 frames, best of 3), at the p50 and p95 frame sizes of the turn goldens and at 4 KiB:

Frame	Unix	TCP	TLS 1.3
120 B	2.9 µs	1.0 µs	17.0 µs
660 B	2.7 µs	1.3 µs	19.2 µs
4 KiB	4.1 µs	2.6 µs	24.6 µs

Python's per-record overhead dominates the TLS column, so it is an upper bound; R7 measures rustls. Even at the upper bound, a turn of 300 to 1,000 frames costs 5 to 25 ms of CPU spread over seconds, and coalescing cuts the records.

- **Round trips.** Pushes are one-way, so the round trip is paid per request, not per frame.

	1 ms RTT	30 ms RTT
Cold <code>theseus ask</code> (TCP 1 + TLS 1.3 1 + <code>turn.submit</code> 1 = 3 RTT before the first event)	+3 ms	+90 ms
Warm TUI or watch (1 RTT)	+1 ms	+30 ms
Each streamed token's delivery	+0.5 ms	+15 ms (not cumulative)
A confirm answer	+1 ms	+30 ms, on top of human time

A model's first token takes roughly 0.5 to 1.5 s, so at 30 ms a cold ask is about 6% to 18% slower to first token and a warm TUI 2% to 6%. At 1 ms neither shows.

- **A remote index needs a persistent connection.** Today's per-call connect under TLS costs 3 RTT a call: 90 ms at 30 ms, which would fail health's 100 ms deadline and eat a third of recall's 250 ms. On one persistent connection a call costs 1 RTT plus compute, and recall keeps about 220 ms. Beyond about 80 ms of RTT the index should sit with the daemon, and health says so.

4. Auth: mTLS with a private CA, device identities, roles

- **Why mTLS over per-client tokens.** A connection is proven before any JSON is parsed (the same pre-parse refusal the web UI's uid check gives); no bearer secret travels in frames, so none can leak into logs or ledgers; a cert names a device, so revocation hits one device; the same machinery covers the index link. A token in a frame is exactly what a scrub has to catch, so tokens are not offered (D2).
- **The CA.** The daemon holds a private CA (P-256). The CA key and the server key live in the secrets board as `op://` refs and resolve at start like every other secret; neither is ever on disk in plaintext. `theseus remote init` (local only) creates them and writes only refs into the config.
- **Identity in place of the peer's uid.** Each client cert carries `device=<name>` and `role=owner|observer|index`. The connection is `Surface::Remote` (never `Cli`) with `Client.label = "remote:<device>"`, so the ledger and every actor string name the device. The owner's identity for remote is "a device with role `owner`".
- **What a remote client may do.**

	owner	observer	index
Read sessions, history, health, watch	yes	yes	no
Read private material (<code>reads_private</code>)	yes (D4)	no: sizes and labels, as Discord-shared	no

	owner	observer	index
turn.submit, open, cancel	yes	no	no
Confirm, trust, untighten, publish, operator methods	yes, ledgered with the device	refused	refused
Local only: shutdown, restart, restore, aws.bootstrap, aws.confirm_alerts, remote.* admin (D5)	no	no	no
wal.follow	no	no	yes, nothing else

- **No host-path reads.** From a remote surface any param that names a daemon-side path is refused; the CLI reads the file locally and sends `text`, as `--attach` and `import` already do.
 - **The gate and confirms are unchanged.** Tools run on the daemon's machine under the same floor, gate and broker. A remote CLI's working directory has no meaning there: turns work in the daemon's workspace (D9).
- **Pairing.**
 1. On the daemon's host, `theseus remote pair <device> --role owner` prints a one-time code (10 minutes, single use, ledgered) and the CA's fingerprint.
 2. On the client, `theseus remote join host:port --code <code>` makes a key and a CSR, pins the CA by that fingerprint, and receives a 30-day cert.
 - The client key goes into the client's own 1Password (`op://`) when `op` is signed in there, otherwise the OS keyring; never a plaintext file (D6). **No client key on the daemon's host, ever:** `join` refuses an address that resolves to the daemon's own machine, whose road is the Unix socket.
 - **Rotation and revocation.** A cert renews itself over the live link at two-thirds of its life (D11). `theseus remote revoke <device>` writes the serial to a revocation list in the store (ledgered) and closes that device's live connections at once. `theseus remote list` shows each device, its role and when it was last seen. Rotating the CA means re-pairing; `remote init --rotate` keeps the old CA trusted for 7 days.

5. Split deployments

- **The index remote from the daemon (D7): the daemon streams the WAL.**
 - `wal.follow { cursor, max_bytes }` sends whole frames from a cursor: `theseus-follow` run on the daemon's side, with its rewind rule and `read_upto`.
 - The remote tender keeps its cursor and index exactly as today and swaps its file follower for a feed follower, `theseus-index serve --feed tls://daemon:7434`. Ingest is already idempotent by node id, and a rewind already means a rebuild.
 - The daemon reaches the tender on one persistent mTLS connection. Supervision becomes health plus backoff (no `--parent`, no SIGTERM); health says `remote`, the lag in bytes and seconds, and the RTT.

- Rejected: the durability tender's S3 copy (5 to 60 s behind, too stale for recall of the current conversation; kept as a cold backfill) and a shared filesystem (inotify does not fire across NFS, and the store must stay local).
- **The trust boundary moves.** The WAL holds everything, private places included, and so does the index's disk, so the index host must be a machine the owner controls. The feed is TLS only, and `index.forget` must be acked by the remote tender before a forget reports done.
- **The CLI remote from the daemon.** `--attach`, `import` and `publish` upload content; no path crosses. `herdr` stays with the CLI. `status` names the daemon's host, and health gains `host`. `--spawn` stays local. Tools run on the daemon; running them on the client's machine ("client hands") is a later design (D9).
- **Several CLIs or TUIs on other machines.** Each has its own device cert, connection, queue and backlog cap. Confirms go to every owner surface, and the first answer wins, as today across the CLI, the web UI and Discord.
- **A daemon on EC2 in the owner's AWS account (D10).** A stack of its own, separate from the hands' network (which has no ingress and must not change: the `network-not-ours` guard, `docs/spec/04-part1-s3.10.md:784`), with an EBS gp3 state volume so the fsync floor holds. Reached through SSM Session Manager port forwarding with **no ingress** (the default), or an allowlisted ingress rule on 7434. Secrets come from an `op` service account on the instance, and durability ships to the same bucket.

6. Failure modes

- **Partitions.** Turns go on in the daemon; clients reconnect and resume from history positions; half-open peers are cut by the ping and the write deadline; a cut-off remote tender leaves recall `unavailable` (the turn goes on) and catches up from its cursor when the link returns.
- **A slow index.** Recall's 250 ms deadline already bounds it; health's 100 ms deadline needs the persistent connection, and past it health shows the last status.
- **Version skew.** `hello` refuses a major mismatch with words naming both builds; minor additions ride `serde` defaults. The web UI's TS client is generated from the same types and served by the daemon, so it is never skewed.
- **Clock skew.** Certs allow 5 minutes either side. Every deadline, cancel and ledger time is the daemon's clock. A client renders the daemon's timestamps in its own zone, so skew moves only "ago" figures; `hello` returns the daemon's time, and the status line warns past 2 s.

7. The live rig

Two network namespaces on one machine joined by a veth pair; `tc qdisc ... netem delay` at 0.5 ms or 15 ms each way gives 1 ms or 30 ms of RTT, and `loss 100%` makes a partition. Root comes through `sudo`. Two real machines come later, for R9.

8. Rows

Every row is a child of `theseus-pp7c`, labelled `stop-point-out`, and **blocked by the benchmark rerun `theseus-lc7a`**: this is a new capability, so the rerun measures Theseus without it. Order: R1 to R7, then R8, then R9 (D12).

#	Beads	Row	Size	What	Gate	Live check
R1	theseus-pp7c.1	line-cap	S	A bounded 64 MiB line reader for every transport (<code>server.rs:138</code>)	Unit tests at the cap ± 1 ; the lifecycle bench unchanged	A 65 MiB line on the Unix socket gets an error frame and a close; the daemon's RSS stays flat
R2	theseus-pp7c.2	wire-hello	S	hello , semver VERSION , host in health; pipelined; required on TCP	Protocol and TS goldens; a skew test	An old-major client refused with both builds named
R3	theseus-pp7c.3	tls-listen	M	[rpc] listen , rustls TLS 1.3, mTLS required, Surface::Remote , the device in Client , ledger rows rpc.remote.connected and refused , write deadline, ping, keepalive, coalesced flush	Units; an rcgen-cert test over loopback; a lifecycle-bench A/B shows the Unix path unchanged	Daemon in netns a, CLI in netns b: theseus --connect asks a turn; a cert from another CA is refused before any JSON is read
R4	theseus-pp7c.4	pairing	M	remote init/pair/join/list/revoke ; CA and server keys as op:// refs; the client key in the client's 1Password or keyring; 30-day renewal; a revocation list in the store	Units; the scrub finds no key material in the ledger or logs	Pair a netns-b client; revoke it while it watches: closed within 1 s; an expired cert refused
R5	theseus-pp7c.5	remote-policy (security review)	M	The role table; path params refused from remote; local-only methods; reads_private per role	A table test over every method in OPERATORS and every path-taking param	An observer's confirm refused; an owner's accepted and ledgered with the device; a remote publish <path> sends text
R6	theseus-pp7c.6	client-connect	S-M	CLI and TUI --connect and THESEUS_ADDR , reconnect and resume, RTT in the status line	A dropped-duplex resume test (no gap, no duplicate)	30 ms RTT plus 1% loss, then a 20 s partition: theseus watch resumes, and its nodes match session.history
R7	theseus-pp7c.7	remote-bench	S	Remote arms in the lifecycle and head-to-head benches at 0, 1 and 30 ms RTT: connect to first event, frames a second, rustls μ s a frame, turn wall time	The report under docs/benchmarks/ with SVGs	The run is the check, and its Unix arm shows no regression
R8	theseus-pp7c.8	wal-feed	L	wal.follow ; theseus-index serve --feed ; [index] remote ; one persistent mTLS daemon-to-tender link; remote supervision in health; forget acked	Follower tests over a streamed feed, rewind included; recall-arm tests	Tender in netns b at 30 ms RTT: the recall arms meet 250 ms; cut mid-batch, it resumes with one copy; a restore makes it rebuild
R9	theseus-pp7c.9	ec2-daemon	L	A daemon stack on EC2 (no-ingress SSM forward, or an allowlisted 7434), gp3 state, an op service account, durability to the same bucket	cfn validate , the guard, accessanalyzer	The owner's laptop CLI drives a turn on the EC2 daemon; the restore drill passes there
R10	theseus-pp7c.10	client-hands	—	Design only: tools that run on the client's machine	—	—

9. The owner's decisions (taken at their defaults, 2026-10-09)

#	Decision	Taken
D1	Amend the reachability rule ("localhost only, no inbound")	Inbound TCP only with mTLS, off unless <code>[rpc] listen</code> names it; the Unix socket stays the default
D2	mTLS or per-client tokens	mTLS; no tokens
D3	The remote framing	NDJSON over TLS; <code>/ws</code> stays browser-only and loopback
D4	May an owner-role remote device read private material and answer confirms?	Yes, owner role only; observers see sizes and labels
D5	Local-only methods	<code>shutdown</code> , <code>restart</code> , <code>restore</code> , <code>aws.bootstrap</code> , <code>aws.confirm_alerts</code> , and the <code>remote.* admin</code>
D6	Where a client's key lives	The client's own 1Password (<code>op://</code>), else the OS keyring; never a plaintext file, never on the daemon's host
D7	How a remote index gets the store	The daemon streams the WAL (<code>wal.follow</code>); the S3 copy only as a cold backfill
D8	Remote web UI or cockpit	No: loopback only; a remote browser uses a forwarded port
D9	Where tools run for a remote CLI	The daemon's machine; client hands are a later design
D10	EC2 daemon reachability	SSM port forwarding, no ingress
D11	Cert lifetime	30 days, auto-renewed at two-thirds of its life
D12	When	After the benchmark rerun (theseus-lc7a): R1 to R7 first, then R8, then R9

Sources outside the repo

The design brief, the TLS microbenchmark script that produced section 3.1's table, and the Discord summary are kept in the working investigations outside this repository. Every code citation above is to this repository at `69cb5375`.